**Вайцуль Александр**

***Задание 1.***

Считаю, что поле *age* не содержит ценной информации. Возраст является динамическим параметром и может изменяться, тогда как значение в базе данных остаётся неизменным. Поэтому более корректно хранить дату рождения клиента в поле *birth_date*.

В текущей структуре отсутствуют поля для связи с клиентом. Необходимо добавить: *email* – для авторизации и коммуникации; *phone* – для SMS-уведомлений и дополнительной идентификации

Можно добавить поле *is_subscribed* типа BOOLEAN для учёта согласия клиента на получение рассылки.

Отсутствует обязательное поле *password_hash* для безопасного хранения хэшированного пароля пользователя.

Поле *city_id* в данной таблице не имеет практического смысла. Информация о местоположении должна храниться в таблице заказов, поскольку: Адрес клиента может изменяться Эти данные не требуются для входа в личный кабинет Такой подход исключит хранение избыточной информации.

Очевидно, что для данной таблице также необходимы ограничения на поля, но это замечание относится уже к физической реализации, а не логического проектирования

**Примерный вид таблицы *clients***

```
CREATE TABLE clients (
    client_id SERIAL PRIMARY KEY,
    first_name VARCHAR(100) NOT NULL,
    last_name VARCHAR(100) NOT NULL,
    birth_date DATE,
    email VARCHAR(255) UNIQUE NOT NULL,
    phone VARCHAR(20) UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    is_subscribed BOOLEAN DEFAULT TRUE);
```

***Задание 2.***

Предлагаю нормализовать структуру данных, разделив таблицу items на две связанные таблицы:

1) items - будет содержать основную информацию о товарах (item_id, item_name)
2) items_price_history - будет хранить историю изменения цен с указанием: price_date - дата изменения цены item_cost - актуальная цена на указанную дату

```
CREATE TABLE items (
        item_id NUMBER PRIMARY KEY,
        item_name VARCHAR(255) NOT NULL
);
-- Таблица истории цен
CREATE TABLE item_prices_history (
        item_id NUMBER REFERENCES items(item_id),
        price_date DATE, -- дата цены
        item_cost NUMBER NOT NULL,  -- Цена в этом месяце
        PRIMARY KEY (item_id, price_date)
);
```

***Задание 3.***

**Исправленный запрос**

```
        SELECT D.DEPARTMENT_ID,
          D.DEPARTMENT_NAME,
          SUM(E.SALARY) AS TOTAL_SALARY
        FROM EMPLOYEES E
        JOIN DEPARTMENTS D ON E.DEPARTMENT_ID = D.DEPARTMENT_ID
        JOIN LOCATIONS L ON D.LOCATION_ID = L.LOCATION_ID
        WHERE E.MANAGER_ID IS NULL
          AND L.CITY = 'SEOUL'
        GROUP BY D.DEPARTMENT_ID, D.DEPARTMENT_NAME
        HAVING SUM(E.SALARY) > 100000;
```

**Ошибки:**

1) Проверка на NULL. Изменил на IS NULL
2) Условия на агрегатные функции должны быть HAVING, а не в WHERE. Неравенство строгое. Следует из условия
3) Соединение таблиц через JOIN
4) В GROUP BY добавлен DEPARTAMENT_ID, так как он есть в SELECT

***Задание 4.***

Потому что его положили рядом со стеной (вплотную).